

Implementation of Synchronous and Asynchronous FIFO using Verilog

Sushil Bahadur Pankaj
Dept. of Computer Science and Engineering
ITER, SOA University
Bhubaneswar, India
sushilbahadurpankaj@gmail.com

Shubham Kumar Sah
Dept. of Electronics and Communication Engineering
ITER, SOA University
Bhubaneswar, India
shubham.sah2212@gmail.com

Abstract—This paper presents the design and implementation of four FIFO (First-In-First-Out) memory of different forms using Verilog HDL: synchronous positive edge, synchronous negative edge, asynchronous positive edge, and asynchronous negative edge. All FIFOs are arranged as 8x8 (8 entries, 8-bit data) and were functionally verified using testbenches in the Vivado simulation environment. The synchronous positive edge FIFO achieved the highest performance at 80 MHz with a setup slack of 22.467 ns and hold slack of 0.089 ns, while consuming 35 LUTs, 84 registers, and 72 mW total power. The synchronous negative edge FIFO operated at 50 MHz with 2.920 ns setup slack, using 36 LUTs and 85 registers. The asynchronous positive edge FIFO successfully transferred data between 66 MHz write and 50 MHz read clocks with 0.935 ns setup slack, utilizing 37 LUTs and 88 registers. The asynchronous negative edge FIFO operated at 33 MHz write and 25 MHz read clocks with 0.253 ns setup slack, using 38 LUTs and 88 registers. All four designs met timing constraints with positive slack and passed functional verification. The results demonstrate that positive edge synchronous FIFO offers the highest performance, while asynchronous FIFOs enable good clock domain crossing at the cost of additional resources and lower operating frequencies. Negative edge triggering consistently resulted in lower maximum frequencies due to FPGA tool optimization preferences for rising-edge clocks.

Index Terms—Synchronous FIFO, Asynchronous FIFO, Positive Edge Triggering, Negative Edge Triggering, Clock Domain Crossing, Verilog HDL, Timing Analysis.

I. INTRODUCTION

In today's digital world, we need to temporarily store data before it can be further processed, for which FIFO (First-In-First-Out) memory buffer comes into play. A FIFO ensures that the very first data written into it should be the very first data to be read out. It can be understood by taking example of a ticket counter where the person who arrives first gets the ticket first. This simple and powerful logic makes FIFO an essential building block of all digital systems today, from embedded systems to processor and communication systems it is used almost everywhere [1].

FIFO helps buffer data in routers and switches, enable smooth communication between sensors, manage video streams in real-time processing systems, and peripherals in embedded systems [2]. Faster components would have to wait constantly for the slower ones if FIFOs were not used, which may lead to inefficiencies and potential data loss. Serving as a

temporary storage for data, FIFO separates the data producer from the consumer, so that each one can operate at its own speed. Key features of FIFO includes full read and write pointer for memory tracking, full and empty flag for flow control and overflow/underflow prevention mechanisms [4].

A. FIFO Classification

FIFO are classified into two types: Synchronous and Asynchronous [3].

Synchronous FIFO: In synchronous FIFO, single clock is used for both write and read operations, which simplifies the design significantly. Due to which there is no risk of metastability, timing constraint is simple, and the full and empty flags are easy to generate. Synchronous FIFOs offer several advantages. They primarily simplify timing closure as they operate under a single clock domain, eliminating the need for complex synchronizers to manage timing mismatches. They provide stable flow control using status flags, ensuring that no invalid operations take place. Their well-organized implementation makes them power- and area-efficient, which is important for resource-constrained and low-power applications.

Asynchronous FIFO: In asynchronous FIFO, two different clocks are used, one to write and another for read operation. It is used when data needs to be transferred between two chips, between a processor and a slow peripheral, or across different clock domain within a large System-on-Chip (SoC). However, working with two different clocks introduces a serious problem of metastability. When a signal changes very close to a clock edge, a flip-flop can enter an unstable state, causing data corruption. First, instead of using regular binary counters for read and write pointers, they use Gray code counters, where only one bit changes at a time. This minimizes switching activity and reduces the probability of metastability. Second, they pass these Gray-coded pointers through dual flip-flop synchronizers before comparing them across clock domains [5].

B. Clock Edge Variations

An important design consideration that is often missed is the choice of clock edge triggering. Most of the time positive edge triggering is used as default because most FPGA and ASIC

tools are optimized for rising edge. However, in certain cases negative edge triggering offers advantages, such as balancing clock load or interfacing with legacy system that uses falling edge. In this paper both the variations were explored to provide a complete comparison. Positive edge triggering means all operation occurs at rising edge of clock and negative edge triggering means all the operation occurs at falling edge of clock.

C. Paper Organization

The remainder of this paper is organized as follows. Section II provides background on FIFO operation and reviews related work from the three reference papers. Section III describes our design methodology for both FIFO types, including RTL implementation details. Section IV presents functional verification, synthesis, timing, and power results. Section V provides a comparative analysis of all four designs. Section VI concludes the paper with key takeaways and directions for future work.

II. BACKGROUND AND RELATED WORK

A. Working of Synchronous and Asynchronous FIFO

This subsection explains the internal operation of both FIFO types, focusing on pointer management, flag generation, and the critical differences between synchronous and asynchronous designs.

1) *Working of Synchronous FIFO*: The operation of synchronous FIFO is controlled by read and write pointer that tracks the memory location for data storage and retrieval. Both pointers work on same clock signal, ensuring consistent timing and eliminating the need for complex clock management [4].

Pointer Initialization and Update: On reset, both the write pointer and read pointer start at zero. Think of them as two arrows pointing to a position in the memory. When write enable is high and FIFO is not full, the data gets stored at the location where the write pointer is currently pointing. The write pointer moves to the next spot once the data is written, ready for the next write. Similarly, when read enable is high the data is taken from the location where the read pointer is pointing, and the read pointer is incremented to the next location. When the pointer reaches the end of memory it simply wraps around to zero. This wrap around behavior allows the FIFO to keep running continuously without needing to manually reset the pointer.

Flag Generation: The FIFO uses two flags: Full and Empty. The FIFO is empty when both pointers point to the same location (wptr equals rptr). The pointers can also be equal when the FIFO is completely full, that is, when the write pointer has gone all the way around and caught up to the read pointer. To tell these two situations apart, we add an extra bit to the pointers. The full condition is detected when the extra bits differ but the rest of the bits match. Once the full flag is set, no more writes are allowed until some data is read. Once the empty flag is set, no more reads are allowed until new data arrives.

Read and Write Timing: All the operations of FIFO occurs at posedge or negedge depending upon the design. The read and write enable signals are sampled at the clock edge, and their corresponding operations are executed on the same clock cycle. The data output becomes valid after the clock edge during a read operation. The critical aspect of the read operation is the order: the data must be read using the current pointer value before the pointer is incremented; otherwise, data will be skipped. The write operation has no such ordering constraint because the write pointer points to the next empty location.

2) *Working of Asynchronous FIFO*: The operation of an asynchronous FIFO is more complex because the write and read operation requires two different clock signals. The key challenge is to accurately transfer pointer values from one clock domain to another without encountering metastability [3].

Pointer Management with Gray Code: Asynchronous FIFOs use Gray code counters instead of binary counters for pointer management. Gray code has the property that only one bit changes between consecutive values, which minimizes the risk of metastability during clock domain crossing. For example, the sequence of 4-bit Gray code values is: 0000, 0001, 0011, 0010, 0110, 0111, 0101, 0100, 1100, 1101, 1111, 1110, 1010, 1011, 1001, 1000. The conversion from binary to Gray code is performed using the formula: $\text{gray} = \text{binary} \wedge (\text{binary} \gg 1)$. The write pointer is maintained in binary, converted to Gray code, and then passed to the read clock domain. Similarly, the read pointer is maintained in binary, converted to Gray code, and passed to the write clock domain.

Synchronization Using Dual Flip-Flops: Before the Gray-coded pointers can be compared across clock domains, they must be synchronized to the destination clock domain to avoid metastability. This is achieved using a dual flip-flop synchronizer, also known as a two-stage synchronizer. The first flip-flop samples the incoming Gray-coded pointer on the destination clock edge. Due to setup or hold time violations, this flip-flop may become metastable. However, the second flip-flop, which samples the output of the first flip-flop after one full clock cycle, provides sufficient time for the metastable signal to resolve to a valid logic level. The probability of metastability propagating through both flip-flops is extremely low, resulting in a high Mean Time Between Failures (MTBF). The cost of this synchronization is a latency of two destination clock cycles for the pointer value to be reliably available.

Full Flag Generation (Write Domain): The full flag is generated in the write clock domain by comparing the write pointer (in Gray code) with a synchronized version of the read pointer. The read pointer (in binary) is first converted to Gray code, then passed through a dual flip-flop synchronizer clocked by the write clock. The synchronized Gray-coded read pointer (rptr_gray_sync) is then compared with the write pointer (wptr_gray). The full condition is detected when the two pointers would be equal if the write pointer were to wrap around. The Gray

code comparison condition is: $\text{full} = (\text{wp_ptr_gray} == \{\sim \text{rp_ptr_gray_sync}[\text{PTR_WIDTH}-1:\text{PTR_WIDTH}-2], \text{rp_ptr_gray_sync}[\text{PTR_WIDTH}-3:0]\})$. This condition effectively checks whether the write pointer has caught up to the read pointer after completing a full cycle.

Empty Flag Generation (Read Domain): The empty flag is generated in the read clock domain by comparing the read pointer (in Gray code) with a synchronized version of the write pointer. The write pointer is converted to Gray code, passed through a dual flip-flop synchronizer clocked by the read clock, and then compared with the read pointer. The empty condition is detected when the two Gray-coded pointers are exactly equal: $\text{empty} = (\text{rp_ptr_gray} == \text{wp_ptr_gray_sync})$. Unlike the full condition, no bit inversion is required because the empty condition occurs when both pointers are at the same address without any wrap-around.

Memory Access: The actual memory array is dual-port, allowing simultaneous read and write operations. The write operation uses the binary write pointer (lower bits only) to address the memory for writing. The read operation uses the binary read pointer (lower bits only) to address the memory for reading. Since the write and read pointers are maintained in their respective clock domains, no synchronization is needed for memory access itself. The memory array operates asynchronously with respect to the two clocks, or it can have separate read and write clock inputs.

III. PROPOSED METHODOLOGY

A. Synchronous FIFO Design

The synchronous FIFO uses a single clock. It has four main parts: write pointer, read pointer, memory array, and flag generator.

Pointers: Both pointers are binary counters. On reset, they start at zero. During a valid write, data goes into the memory location pointed by the write pointer, then the pointer increments. During a valid read, data comes from the location pointed by the read pointer, then that pointer increments. When a pointer reaches the end, it wraps to zero.

Memory: A register array of size $\text{DEPTH} \times \text{WIDTH}$ stores the data. Writes use the write pointer as address; reads use the read pointer.

Flags: A separate counter tracks how many items are in the FIFO. It increments on writes and decrements on reads. Full is set when counter equals DEPTH. Empty is set when counter equals zero.

B. Asynchronous FIFO Design

The asynchronous FIFO uses two independent clocks: one for writes, one for reads. It adds Gray code and synchronizers to safely move pointers between clock domains.

Gray Code: Pointers are converted from binary to Gray code using: $\text{gray} = \text{binary} \wedge (\text{binary} \gg 1)$. Gray code changes only one bit at a time, which reduces the risk of metastability.

Synchronizer: Two flip-flops in series, both clocked by the destination clock. The first flip-flop may become unstable, but

the second one samples after one full cycle, giving time for the signal to settle.

Full Flag (write domain): Compares write Gray pointer with synchronized read Gray pointer. The condition is: $\text{wp_ptr_gray} == \{\sim \text{rp_ptr_gray_sync}[\text{MSB}:\text{MSB}-1], \text{rp_ptr_gray_sync}[\text{MSB}-2:0]\}$.

Empty Flag (read domain): Compares read Gray pointer with synchronized write Gray pointer. The condition is simply: $\text{rp_ptr_gray} == \text{wp_ptr_gray_sync}$.

Memory: Dual-port RAM allows simultaneous read and write from different clock domains.

C. Clock Edge Variations

Both FIFO types were implemented in two versions: positive-edge and negative-edge triggered.

Positive-Edge: All flip-flops sample on the rising clock edge (0→1). This is the default and better optimized by tools. Always blocks use $@(\text{posedge } \text{clk})$.

Negative-Edge: Flip-flops sample on the falling edge (1→0). Tools are less optimized for this, leading to lower maximum frequencies. Always blocks use $@(\text{negedge } \text{clk})$.

Target Frequencies:

- Sync posedge: 80 MHz
- Sync negedge: 50 MHz
- Async posedge: 66 MHz (write), 50 MHz (read)
- Async negedge: 33 MHz (write), 25 MHz (read)

IV. IMPLEMENTATION DETAILS

A. RTL Description

The synchronous FIFO uses a single always block where write and read operations occur on the same clock edge. A counter tracks the number of valid entries. The full flag is set when the counter equals the FIFO depth, and the empty flag is set when the counter equals zero.

The asynchronous FIFO uses separate always blocks for the write and read domains. Write and read pointers are converted to Gray code to minimize switching activity. Dual-flip-flop synchronizers transfer these Gray-coded pointers between clock domains. The full flag is generated in the write domain by comparing the write pointer with the synchronized read pointer. The empty flag is generated in the read domain by comparing the read pointer with the synchronized write pointer.

B. Testbench Design

All testbenches follow the same structure. Clocks are generated using forever loops. Reset is applied for 2-3 cycles at the start. The test procedure writes 8 values into the FIFO and then reads them back. Each read value is compared with the expected value, and a PASS or FAIL message is displayed.

C. Constraint Development

Timing constraints were written in XDC format. Clock constraints define the target frequencies for each design. For the synchronous FIFO, a single clock constraint is used. For the asynchronous FIFO, separate constraints are defined

for the write and read clocks, and they are declared as an asynchronous clock group. Input and output delays are set to 2.0 ns and 2.5 ns respectively. Reset signals are set as false paths.

V. RESULTS AND ANALYSIS

A. Functional Verification Results

All four FIFO designs passed functional verification. Each testbench wrote 8 values (1 through 8) into the FIFO and read them back. For the synchronous FIFO, all read values matched the expected values in correct order. The full flag asserted after the eighth write, and the empty flag asserted after the eighth read. For the asynchronous FIFO, the same test was performed with independent write and read clocks. All 8 values were successfully transferred across clock domains without data loss or corruption. The simulation waveforms for all four designs are shown in Figures 1 through 4.

B. Resource Utilization

All designs were synthesized for an Artix-7 FPGA. The synchronous positive-edge FIFO used 35 LUTs and 84 registers. The synchronous negative-edge FIFO used 36 LUTs and 85 registers. The asynchronous positive-edge FIFO used 37 LUTs and 88 registers. The asynchronous negative-edge FIFO used 38 LUTs and 88 registers. All designs used 22 I/O pins except the asynchronous versions which used 24 I/O pins and 2 BUFGCTRLs. The resource utilization reports for all four designs are presented in Figures 5 through 8.

C. Timing Analysis

All four designs met timing constraints with positive slack. The synchronous positive-edge FIFO achieved 80 MHz with a setup slack of 22.467 ns and hold slack of 0.089 ns, as shown in Figure 9. The synchronous negative-edge FIFO achieved 50 MHz with a setup slack of 2.920 ns and hold slack of 0.145 ns (Figure 10). The asynchronous positive-edge FIFO operated at 66 MHz write and 50 MHz read with a setup slack of 0.935 ns and hold slack of 0.166 ns (Figure 11). The asynchronous negative-edge FIFO operated at 33 MHz write and 25 MHz read with a setup slack of 0.253 ns and hold slack of 0.140 ns (Figure 12).

D. Power Analysis

Total on-chip power was 72 mW for all four designs, as shown in Figures 13 through 16. Static power dominated at 70 mW (98%), while dynamic power contributed only 1-2 mW (2%). Dynamic power was primarily consumed by I/O pins (67-72%), followed by clocks, signals, and logic. The junction temperature was estimated at 25.4°C with a thermal margin of 59.6°C.

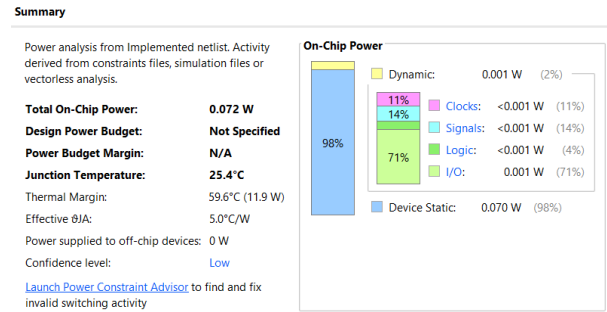


Fig. 13. Power Analysis for Synchronous Positive Edge FIFO

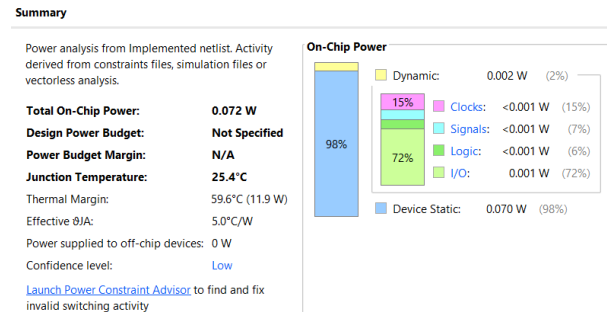


Fig. 14. Power Analysis for Synchronous Negative Edge FIFO

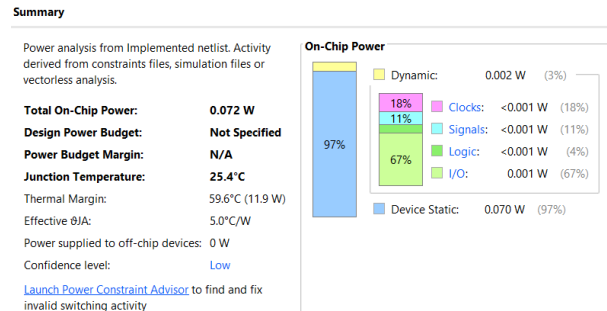


Fig. 15. Power Analysis for Asynchronous Positive Edge FIFO

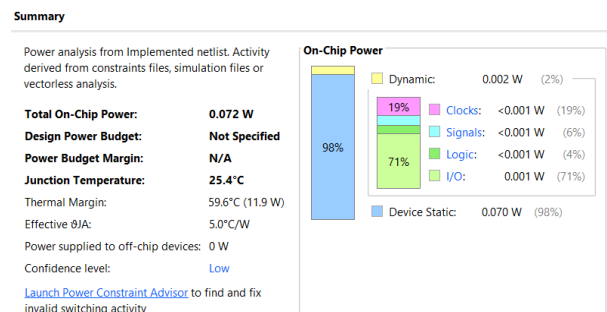


Fig. 16. Power Analysis for Asynchronous Negative Edge FIFO

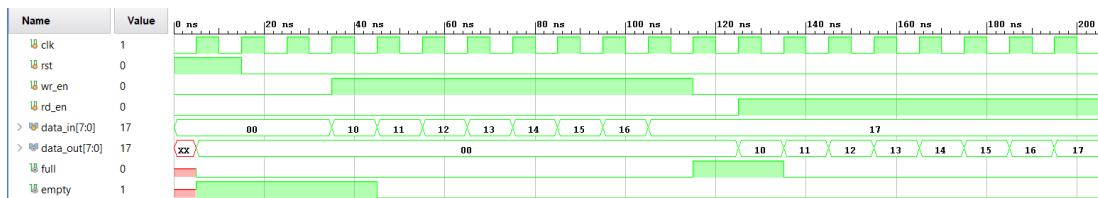


Fig. 1. Synchronous Positive Edge FIFO Simulation Waveform

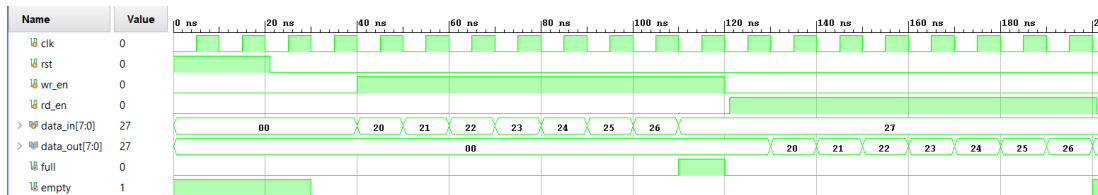


Fig. 2. Synchronous Negative Edge FIFO Simulation Waveform

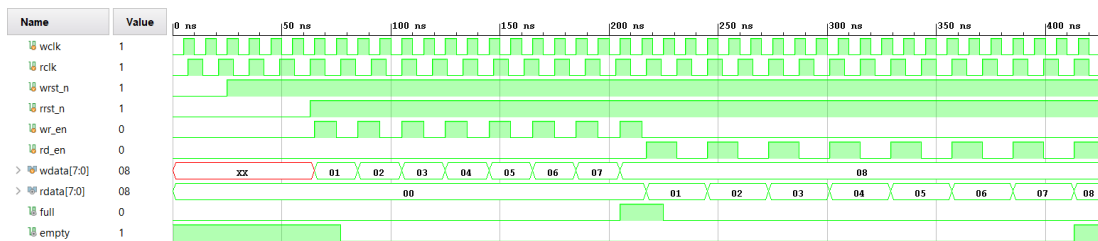


Fig. 3. Asynchronous Positive Edge FIFO Simulation Waveform

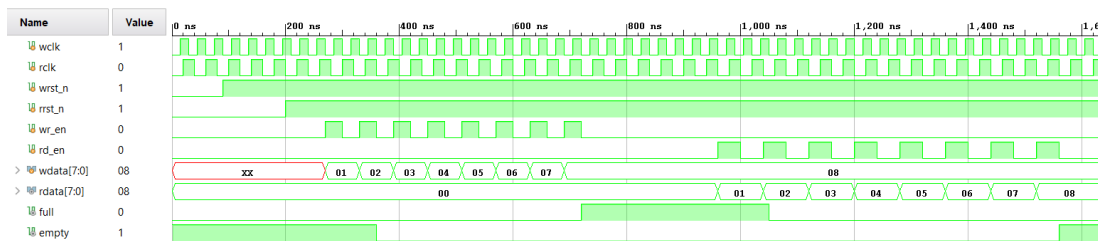


Fig. 4. Asynchronous Negative Edge FIFO Simulation Waveform

Name	Slice LUTs (20800)	Bonded IOB (106)	BUFGCTRL (32)	Slice Registers (41600)	F7 Muxes (16300)	Slice (8150)	LUT as Logic (20800)
sync_fifo_posedge	35	22	1	84	8	19	35

Fig. 5. Resource Utilization for Synchronous Positive Edge FIFO

Name	Slice LUTs (20800)	Bonded IOB (106)	BUFGCTRL (32)	Slice Registers (41600)	F7 Muxes (16300)	Slice (8150)	LUT as Logic (20800)
sync_fifo_negedge	36	22	1	85	8	26	36

Fig. 6. Resource Utilization for Synchronous Negative Edge FIFO

Name	Slice LUTs (20800)	Bonded IOB (106)	BUFGCTRL (32)	Slice Registers (41600)	F7 Muxes (16300)	Slice (8150)	LUT as Logic (20800)
async_fifo_posedge	37	24	2	88	8	25	37

Fig. 7. Resource Utilization for Asynchronous Positive Edge FIFO

Name	Slice LUTs (20800)	Bonded IOB (106)	BUFGCTRL (32)	Slice Registers (41600)	F7 Muxes (16300)	Slice (8150)	LUT as Logic (20800)
async_fifo_negedge	38	24	2	88	8	29	38

Fig. 8. Resource Utilization for Asynchronous Negative Edge FIFO

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 22.467 ns	Worst Hold Slack (WHS): 0.089 ns	Worst Pulse Width Slack (WPWS): 19.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 166	Total Number of Endpoints: 166	Total Number of Endpoints: 85

All user specified timing constraints are met.

Fig. 9. Timing Summary for Synchronous Positive Edge FIFO

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 2.920 ns	Worst Hold Slack (WHS): 0.145 ns	Worst Pulse Width Slack (WPWS): 9.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 171	Total Number of Endpoints: 171	Total Number of Endpoints: 86

All user specified timing constraints are met.

Fig. 10. Timing Summary for Synchronous Negative Edge FIFO

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.935 ns	Worst Hold Slack (WHS): 0.166 ns	Worst Pulse Width Slack (WPWS): 7.000 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 194	Total Number of Endpoints: 194	Total Number of Endpoints: 90

All user specified timing constraints are met.

Fig. 11. Timing Summary for Asynchronous Positive Edge FIFO

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.253 ns	Worst Hold Slack (WHS): 0.140 ns	Worst Pulse Width Slack (WPWS): 14.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 194	Total Number of Endpoints: 194	Total Number of Endpoints: 90

All user specified timing constraints are met.

Fig. 12. Timing Summary for Asynchronous Negative Edge FIFO

VI. COMPARATIVE ANALYSIS

A. Synchronous vs Asynchronous FIFO

Synchronous FIFOs offer higher performance, lower area, and simpler design. They achieve up to 80 MHz compared to 66 MHz for asynchronous FIFOs. However, they cannot be used for clock domain crossing. Asynchronous FIFOs enable reliable data transfer between different clock domains at the cost of additional resources (37-38 LUTs vs 35-36 LUTs) and lower maximum frequencies. They also require Gray code converters and dual-flip-flop synchronizers, increasing design complexity. For applications where both sides operate on the same clock, synchronous FIFOs are the better choice. For systems with multiple clock domains, asynchronous FIFOs are essential despite their higher resource usage.

B. Positive Edge vs Negative Edge Triggering

Positive-edge triggering consistently outperformed negative-edge triggering across both synchronous and asynchronous designs. The synchronous positive-edge FIFO achieved 80 MHz while the negative-edge version reached only 50 MHz. Similarly, the asynchronous positive-edge FIFO operated at 66/50 MHz while the negative-edge version operated at 33/25 MHz. This performance gap exists because FPGA synthesis tools are heavily optimized for rising-edge clocks. Negative-edge designs also consumed slightly more LUTs (36-38 vs 35-37) due to additional logic required for hold time fixes. Therefore, positive-edge triggering is recommended for most applications unless there is a specific requirement for negative-edge operation.

C. Comparison of All Four FIFO Variations

Table I summarizes the key results for all four FIFO designs implemented in this work. The table compares timing, resource utilization, and power consumption across synchronous and asynchronous designs with both clock edge variations.

The results demonstrate that the synchronous positive-edge FIFO achieves the highest performance with the best setup slack of 22.467 ns. The synchronous negative-edge FIFO operates at a lower frequency of 50 MHz but maintains positive timing slack. The asynchronous FIFOs successfully enable clock domain crossing, with the positive-edge version achieving higher write frequencies (66 MHz) compared to the negative-edge version (33 MHz). Resource utilization increases progressively from synchronous posedge (35 LUTs) to asynchronous negedge (38 LUTs), reflecting the increasing complexity of the designs. Power consumption remains constant at 72 mW across all four variants, with static power dominating at 97-98%. This indicates that the additional logic in asynchronous and negative-edge designs does not significantly impact overall power consumption for this small-scale implementation.

D. Comparison with Existing Work

Table II compares this work with the three reference papers. Unlike previous works that focused on either synchronous or asynchronous FIFOs in isolation, this paper provides a

complete comparison of both FIFO types and both clock edge variations. Quantitative timing, area, and power results are provided for all four designs, which were missing in most existing literature. The negative-edge analysis presented here is also absent in the reference papers.

VII. CONCLUSION AND FUTURE WORK

A. Summary of Contributions

This paper presented the design and implementation of four FIFO variants: synchronous positive-edge, synchronous negative-edge, asynchronous positive-edge, and asynchronous negative-edge. All designs were functionally verified and met timing constraints on an Artix-7 FPGA. The key contributions are summarized below:

- 1) The synchronous positive-edge FIFO achieved the highest performance at 80 MHz with 35 LUTs, 84 registers, and 72 mW power.
- 2) The synchronous negative-edge FIFO operated at 50 MHz with 36 LUTs and 85 registers.
- 3) The asynchronous positive-edge FIFO successfully transferred data between 66 MHz write and 50 MHz read clocks with 37 LUTs and 88 registers.
- 4) The asynchronous negative-edge FIFO operated at 33 MHz write and 25 MHz read clocks with 38 LUTs and 88 registers.
- 5) Positive-edge designs consistently outperformed negative-edge designs due to better tool optimization.
- 6) Synchronous FIFOs are more resource-efficient than asynchronous FIFOs but cannot handle clock domain crossing.

B. Limitations and Challenges

The following limitations were encountered during this work:

- 1) The asynchronous negative-edge FIFO required the lowest operating frequencies (33/25 MHz) due to the combined overhead of clock domain crossing and negative-edge optimization.
- 2) Hold timing violations were more frequent in negative-edge designs, requiring additional constraints and margin.
- 3) The designs were verified through simulation only and not tested on actual FPGA hardware.
- 4) The depth and width were fixed at 8x8 for comparison; larger configurations were not explored.
- 5) UVM-based verification, which is industry standard, was not implemented due to time constraints.

C. Future Directions

Several directions for future work are identified:

- 1) Implement the designs on actual FPGA hardware and validate with real-time testing.
- 2) Add threshold flags (almost full and almost empty) to provide early warning to the system.
- 3) Implement fall-through mode to reduce latency in asynchronous FIFOs.

TABLE I
COMPARISON OF ALL FOUR FIFO VARIATIONS

Parameter	Sync Posedge	Sync Negedge	Async Posedge	Async Negedge
Timing				
Write Clock Frequency	80 MHz	50 MHz	66 MHz	33 MHz
Read Clock Frequency	80 MHz	50 MHz	50 MHz	25 MHz
Setup Slack	22.467 ns	2.920 ns	0.935 ns	0.253 ns
Hold Slack	0.089 ns	0.145 ns	0.166 ns	0.140 ns
Resource Utilization				
Slice LUTs	35	36	37	38
Slice Registers	84	85	88	88
Bonded IOB	22	22	24	24
BUFGCTRL	1	1	2	2
Power Analysis				
Total Power	72 mW	72 mW	72 mW	72 mW
Dynamic Power	1 mW (2%)	2 mW (2%)	2 mW (3%)	2 mW (2%)
Static Power	70 mW (98%)	70 mW (98%)	70 mW (97%)	70 mW (98%)
Junction Temperature	25.4°C	25.4°C	25.4°C	25.4°C
Thermal Margin	59.6°C	59.6°C	59.6°C	59.6°C

TABLE II
COMPARISON WITH EXISTING WORK

Feature	Penta & Islam	Yadlapati & Kakarla	Himanshu & Charan	This Work
FIFO Type	Sync	Async	Async	Both
Clock Edges	Posedge only	Not specified	Mixed	Posedge & Negedge
Depth/Width	8x8	Not specified	16 bytes	8x8
Gray Code	No	Yes	Yes	Yes
Timing Results	Limited	No	No	Yes
Power Results	Yes	No	No	Yes
Area Results	Yes	No	No	Yes
Negedge Analysis	No	No	No	Yes

- 4) Explore larger depth configurations (16, 32, 64 entries) and analyze scaling behavior.
- 5) Develop UVM-based verification environment for more rigorous testing.
- 6) Investigate mixed-edge designs where write and read use different clock edges.
- 7) Compare power consumption across different FPGA families and technology nodes.

REFERENCES

- [1] E. Xie and J. Zhou, "Analysis and Comparison of Asynchronous FIFO and Synchronous FIFO," in *2023 IEEE 2nd International Conference on Electrical Engineering, Big Data and Algorithms (EEBDA)*, Changchun, China, 2023, pp. 260-264, doi: 10.1109/EEBDA56825.2023.10090586.
- [2] A. Ghosh and D. Mitra, "A comparative study of asynchronous and synchronous FIFOs for embedded systems," *Microelectronics Journal*, vol. 134, p. 105872, 2023, doi: 10.1016/j.mejo.2023.105872.
- [3] Avinash Yadlapati and Hari Kishore Kakarla, "Design and Verification of Asynchronous FIFO with Novel Architecture Using Verilog HDL," *Journal of Engineering and Applied Sciences*, vol. 14, no. 1, pp. 159-163, 2019.
- [4] Yatheeswar Penta and Riadul Islam, "Design and Verification of a Synchronous First In First Out (FIFO)," Technical Report, 2024.
- [5] Himanshu and Chaagan Charan, "A 16-byte Asynchronous Gray Code FIFO Memory using Verilog HDL for Real Time Applications," in *Proceedings of National Institute of Technology Kurukshetra Conference*, 2024.